

To-Do App Project

Build a Kanban-style task board with drag-and-drop, powered by HTML, Tailwind CSS, JavaScript and a simple JSON backend.

WHAT WE COVER

HTML

Tailwind CSS

JavaScript

Drag & drop

JSON Server

This project ties together everything from the roadmap so far. You'll build a real Kanban board — three columns (To Do, In Progress, Done) — where you drag task cards between columns and the change is saved to a backend. Follow the steps in order; every line of code you need is right here in this guide.

To-Do Application Project

We'll build the board layout with Tailwind, fill it with tasks fetched from a backend, then wire up native HTML drag-and-drop so moving a card updates its status — and saves it. Work through each step and test as you go.

PROJECT CONTENTS

- | | |
|---|--|
| 01. Introduction & what you'll build | 06. Step 3 — Fetch & display tasks |
| 02. Tools & technologies | 07. Step 4 — Drag-and-drop & status updates |
| 03. Setup: Tailwind, structure & JSON Server | 08. Step 5 — Test & debug |
| 04. Step 1 — The HTML structure | 09. Backend options: Placeholder vs Server |
| 05. Step 2 — Style it with Tailwind | 10. Conclusion & enhancements |

01 Introduction & what you'll build

A **Kanban board** is a visual to-do list split into columns by status. You move a card from one column to the next as work progresses. Trello is the famous example.

By the end you'll have a working board with three columns — **To Do**, **In Progress**, **Done** — where dragging a task card to another column updates its status and saves it to a backend so it survives a page refresh.

- ▶ A responsive three-column layout built with Tailwind.
- ▶ Task cards loaded from a backend with `fetch()`.
- ▶ Native drag-and-drop to move cards between columns.
- ▶ Each move sent back to the server so the change persists.

02 Tools & technologies

- ▶ **HTML** — the board and column structure.
- ▶ **Tailwind CSS** (via CDN) — all styling.
- ▶ **JavaScript** — fetching data, rendering cards, drag-and-drop logic.
- ▶ **JSON Server** — a tiny fake REST API that reads/writes a local `db.json` file, so status changes actually save.
- ▶ **Node.js & npm** — needed only to run JSON Server.

No framework, no build tools — just a browser and one small backend. We use the `fetch` API and native drag events you learned earlier.

03 Setup: Tailwind, structure & JSON Server

1. Project structure

Create a project folder with two files. That's all the front end needs.

```
todo-app/
├─ index.html    <!-- markup + Tailwind CDN + our script -->
└─ db.json       <!-- the data JSON Server serves -->
```

TERMINAL

2. Add Tailwind (CDN)

Tailwind loads from a single script tag — no install needed for the front end.

```
<script src="https://cdn.tailwindcss.com"></script>
```

HTML

3. Install & start JSON Server

JSON Server turns a JSON file into a full REST API. Install it once, then point it at your data file.

```
# install once, globally
npm install -g json-server

# from inside the todo-app folder, start it on port 3000
json-server --watch db.json --port 3000
```

TERMINAL

Seed `db.json` with a few tasks. Each task has an `id`, a `title`, and a `status` matching a column.

```
{
  "tasks": [
    { "id": 1, "title": "Design the landing page", "status": "todo" },
    { "id": 2, "title": "Set up the database",      "status": "inprogress" },
    { "id": 3, "title": "Write the documentation", "status": "done" }
  ]
}
```

JSON

JSON Server now gives you `GET /tasks`, `POST /tasks`, and `PATCH /tasks/:id` at `http://localhost:3000`.

04 Step 1 — The HTML structure

Build the page skeleton: a heading and three columns. Each column has a title and an empty container (with an `id` matching a status) that JavaScript will fill.

HTML

```

<body>
  <div class="min-h-screen bg-gray-100 p-6">
    <h1 class="text-2xl font-bold text-gray-800 mb-6">My Kanban Board</h1>

    <div class="grid grid-cols-1 md:grid-cols-3 gap-6">

      <div class="bg-gray-200 rounded-xl p-4">
        <h2 class="font-semibold text-gray-700 mb-3">To Do</h2>
        <div id="todo" class="task-list space-y-3 min-h-[120px]"></div>
      </div>

      <div class="bg-gray-200 rounded-xl p-4">
        <h2 class="font-semibold text-gray-700 mb-3">In Progress</h2>
        <div id="inprogress" class="task-list space-y-3 min-h-[120px]"></div>
      </div>

      <div class="bg-gray-200 rounded-xl p-4">
        <h2 class="font-semibold text-gray-700 mb-3">Done</h2>
        <div id="done" class="task-list space-y-3 min-h-[120px]"></div>
      </div>

    </div>
  </div>

  <script src="app.js"></script> <!-- or an inline <script> tag -->
</body>

```

- ▶ The three container `id`s — `todo`, `inprogress`, `done` — exactly match the `status` values in `db.json`. That link is what makes rendering and dropping simple.
- ▶ `class="task-list"` lets us grab all three columns at once in JavaScript.

05 Step 2 — Style it with Tailwind

Most styling is already done inline with Tailwind classes in Step 1 — that's the utility-first approach. Here's what those classes are doing:

- ▶ `min-h-screen bg-gray-100 p-6` — full-height page, light background, padding.
- ▶ `grid grid-cols-1 md:grid-cols-3 gap-6` — one column on phones, three on desktop.
- ▶ `bg-gray-200 rounded-xl p-4` — each column is a soft rounded panel.
- ▶ `space-y-3` — vertical spacing between the cards inside a column.

The task cards themselves get created in JavaScript, so we'll give them their Tailwind classes there in the next step.

06 Step 3 — Fetch & display tasks

Now the JavaScript. First, fetch the tasks from JSON Server, then place each card into the column that matches its status.

```

const API = "http://localhost:3000/tasks";

async function loadTasks() {
  const res = await fetch(API);
  const tasks = await res.json();
  renderTasks(tasks);
}

function renderTasks(tasks) {
  // empty every column first, so we don't duplicate cards
  document.querySelectorAll(".task-list").forEach(col => (col.innerHTML = ""));

  tasks.forEach(task => {
    const card = createCard(task);
    document.getElementById(task.status).appendChild(card); // status = column id
  });
}

function createCard(task) {
  const card = document.createElement("div");
  card.className = "bg-white p-3 rounded-lg shadow cursor-move";
  card.textContent = task.title;
  card.draggable = true;
  card.dataset.id = task.id; // remember which task this is
  return card;
}

loadTasks(); // run it on page load

```

Open the page and you'll see the three seed tasks land in their columns. `task.status` is used directly as the column `id` — that's why the names had to match.

07 Step 4 — Drag-and-drop & status updates

This is the heart of the app. Native HTML5 drag-and-drop needs three events: **dragstart** (remember what's being dragged), **dragover** (allow the drop), and **drop** (move it and save).

Remember the card being dragged

Add this inside `createCard`, so each card announces its id when you start dragging it:

```

card.addEventListener("dragstart", (e) => {
  e.dataTransfer.setData("text/plain", task.id); // pass the id to the drop
});

```

Accept drops on each column & save the change

```
document.querySelectorAll(".task-list").forEach(column => {
  // allow dropping (by default, elements reject drops)
  column.addEventListener("dragover", (e) => e.preventDefault());

  column.addEventListener("drop", async (e) => {
    e.preventDefault();
    const id = e.dataTransfer.getData("text/plain");
    const newStatus = column.id; // "todo" | "inprogress" | "done"

    // save the new status to the backend
    await fetch(`${API}/${id}`, {
      method: "PATCH",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ status: newStatus })
    });

    loadTasks(); // re-render so the card appears in its new column
  });
});
```

JS

- ▶ `dragover` + `e.preventDefault()` is required — without it the browser won't allow a drop.
- ▶ `drop` reads the dragged id, sets the new status from the column's own `id`, and PATCHes it to the server.
- ▶ Calling `loadTasks()` again re-draws the board from the saved data — simple and always correct.

08 Step 5 — Test & debug

- ▶ **Is JSON Server running?** Visit `http://localhost:3000/tasks` in the browser — you should see your JSON.
- ▶ **Cards not showing?** Open DevTools → Console for errors, and check a `status` value isn't misspelled (must be `todo` / `inprogress` / `done`).
- ▶ **Drop does nothing?** Make sure you called `e.preventDefault()` in **both** the dragover and drop handlers.
- ▶ **Change didn't save?** Open DevTools → Network, drop a card, and confirm the `PATCH` request returns 200.
- ▶ **CORS or connection error?** Confirm the API URL and port match where JSON Server is running.

Refresh the page after moving a card — if it stays in its new column, persistence is working end to end.

09 Backend options: Placeholder vs Server

JSON Server (recommended for this project)

Because our board needs to **save** status changes, JSON Server is the right fit. It supports `PATCH` and writes changes to `db.json`, so moves survive a refresh. Everything above assumes JSON Server.

JSONPlaceholder (read-only alternative)

jsonplaceholder.typicode.com is a free online fake API — no install needed. Great for practising the fetch-and-display part:

```
const res = await fetch("https://jsonplaceholder.typicode.com/todos?_limit=6");
const todos = await res.json(); // [{ id, title, completed }, ...]
```

JS

- ▶ Good for: loading and displaying tasks quickly without any setup.
- ▶ Limitation: it **accepts** a `PATCH` and returns a fake success, but does **not** actually save — a refresh loses your change.
- ▶ So use JSONPlaceholder to practise rendering; switch to JSON Server when you want drag-and-drop to persist.

Conclusion & enhancements

You built a working Kanban board: a Tailwind layout, tasks fetched from a backend, drag-and-drop between columns, and status changes saved through a REST API. That's the same core loop — **fetch** → **render** → **act** → **save** — behind most real web apps.

Ideas to take it further:

- ▶ Add a form to create new tasks (`POST /tasks`).
- ▶ Add a delete button on each card (`DELETE /tasks/:id`).
- ▶ Show a coloured badge or highlight the column you're dragging over.
- ▶ Add priorities, due dates, or a search filter.
- ▶ Rebuild the UI as React components once you learn React.

QUICK RECAP

Layout with **Tailwind** · load tasks with **fetch()** · **drag-and-drop** uses `dragstart` / `dragover` / `drop` · a **PATCH** to **JSON Server** saves the new status. Fetch → render → act → save.